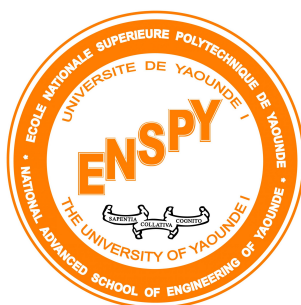




École Nationale Supérieure Polytechnique de Yaoundé
National Advanced School of Engineering of Yaounde

Département de Génie Informatique
Departement of Computer Engineering

U.E : INTRODUCTION AUX RÉSEAUX INFORMATIQUES

**PROJET DE RÉSEAUX :
SEARCH WEB RESPONSIVE**

Rapport de Projet

Réalisé par les étudiants :

EWANE BATCHAKUI NEHEMIE TURING	22P539
TAMEGUE NEGOU DONALD	22P962
NTIH TCHIO TAMOGOU DARYL	22P250
KAYA TEGUIM DESMOND	24P755
KAMDEM KOUAM EZECHIEL JOVAN	22P594

Niveau: III

Sous la supervision de:

Prof. Thomas Djotio Ndié

Année académique: 2024-2025

CONTENTS

LIST OF FIGURES

Dans le cadre de notre projet de réseaux de 3e année, il nous a été assigné le thème suivant : "SEARCH WEB RESPONSIVE". Ce projet a pour objectif de concevoir et de développer un moteur de recherche web intégré à une application responsive fédérant plusieurs services. Il s'agit de permettre à plusieurs utilisateurs de rechercher efficacement des informations ou des fonctionnalités au sein de ces services, tout en assurant une authentification complète (inscription, connexion) au système.

Dans un contexte où les applications numériques offrent une multitude de services, la capacité à centraliser l'accès à l'information et à simplifier l'interaction utilisateur devient un enjeu crucial. Ce projet s'inscrit dans cette dynamique en proposant une solution moderne, basée notamment sur Elasticsearch pour l'indexation et la recherche rapide, et sur une interface web intuitive qui facilite l'expérience utilisateur. L'authentification sécurisée permet quant à elle de garantir la personnalisation des services et la traçabilité des accès.

Le présent document expose les détails et spécifications du projet, la démarche de conception adoptée, les choix technologiques effectués, les différentes étapes de mise en œuvre, ainsi que les résultats obtenus.

CHAPTER 1

PRESENTATION DU PROJET

1.1 Problématique

Dans un contexte numérique marqué par la multiplication des services, des données et des contenus, les utilisateurs sont souvent confrontés à une difficulté majeure : retrouver rapidement une information pertinente parmi un grand volume de ressources disponibles. Cette dispersion de l'information nuit à l'efficacité de l'expérience utilisateur, ralentit l'accès aux fonctionnalités essentielles et rend l'interaction avec les systèmes complexes.

Comment alors centraliser, indexer et restituer efficacement les informations issues de plusieurs services fédérés, tout en garantissant la pertinence des résultats, la sécurité des accès et la simplicité d'utilisation ?

1.2 Objectif du projet

Les différents objectifs que nous cherchons à atteindre au terme de ce projet sont :

Centraliser l'accès à l'information : Offrir un point d'entrée unique permettant de rechercher des contenus ou des services issus de plusieurs modules ou services fédérés de l'application.

Assurer la pertinence des résultats Fournir des résultats de recherche adaptés aux besoins de l'utilisateur, grâce à des techniques d'indexation et de classement basées sur la pertinence des données (par exemple via Elasticsearch).

Améliorer l'expérience utilisateur Simplifier et accélérer la navigation dans l'application en permettant à l'utilisateur de trouver ce qu'il cherche sans avoir à parcourir manuelle-

ment chaque section.

Gérer l'authentification des utilisateurs Intégrer des fonctionnalités d'inscription, de connexion et de gestion des profils afin de personnaliser les résultats de recherche et sécuriser l'accès aux ressources.

1.3 Étude de l'existant

Les moteurs de recherche comme Google ou Bing reposent sur trois étapes principales : exploration des sites web (par des crawlers), indexation des contenus, puis recherche avec calcul de pertinence. Ces systèmes utilisent souvent des crawlers internes pour parcourir tout le Web.

Dans notre cas, l'approche est adaptée au contexte distribué. Les sites des clients Classe A ne sont pas accessibles publiquement. Ainsi, au lieu de crawler à distance, un agent d'exploration est installé localement chez le client. Cet agent analyse dynamiquement le contenu de leur site (même s'il est généré par JavaScript), puis envoie les données nécessaires à notre serveur central pour indexation et recherche ultérieure.

Cette architecture hybride permet de garantir la confidentialité, la scalabilité et l'efficacité de l'indexation, tout en gardant un fonctionnement similaire aux grands moteurs (crawler + index + moteur de recherche).

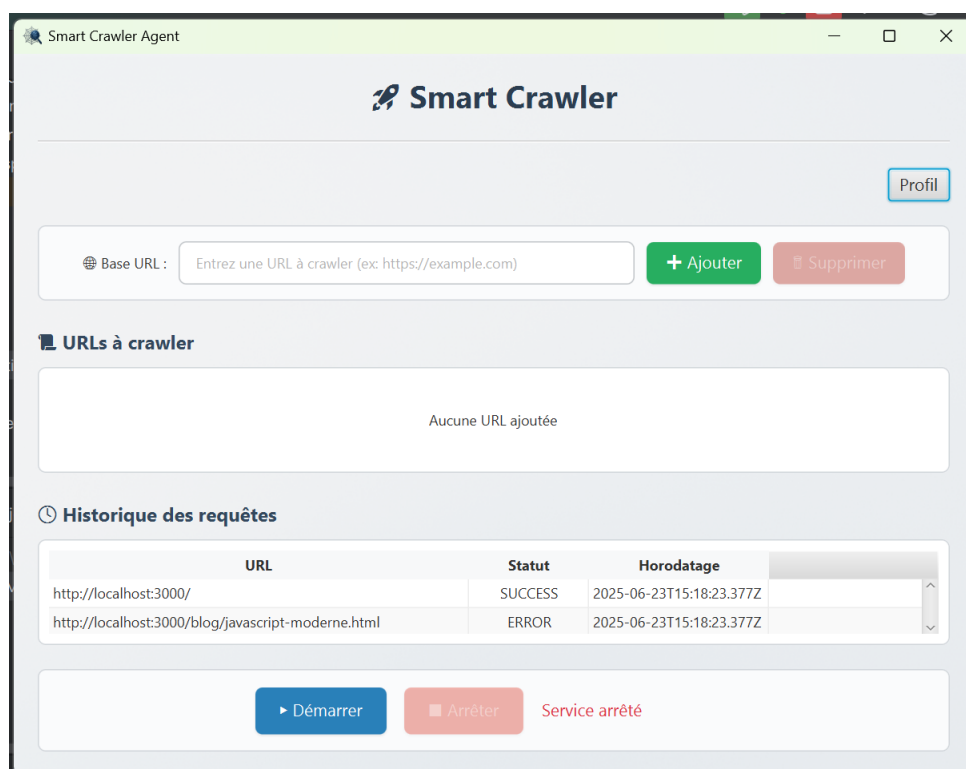


Figure 1.1: Architecture de l'agent d'exploration installé chez le client

L'agent d'exploration, une fois installé, effectue les tâches suivantes :

- Analyse du contenu statique et dynamique du site client.
- Transmission sécurisée des données vers le serveur central.
- Mise à jour périodique des contenus indexés pour refléter les modifications du site.

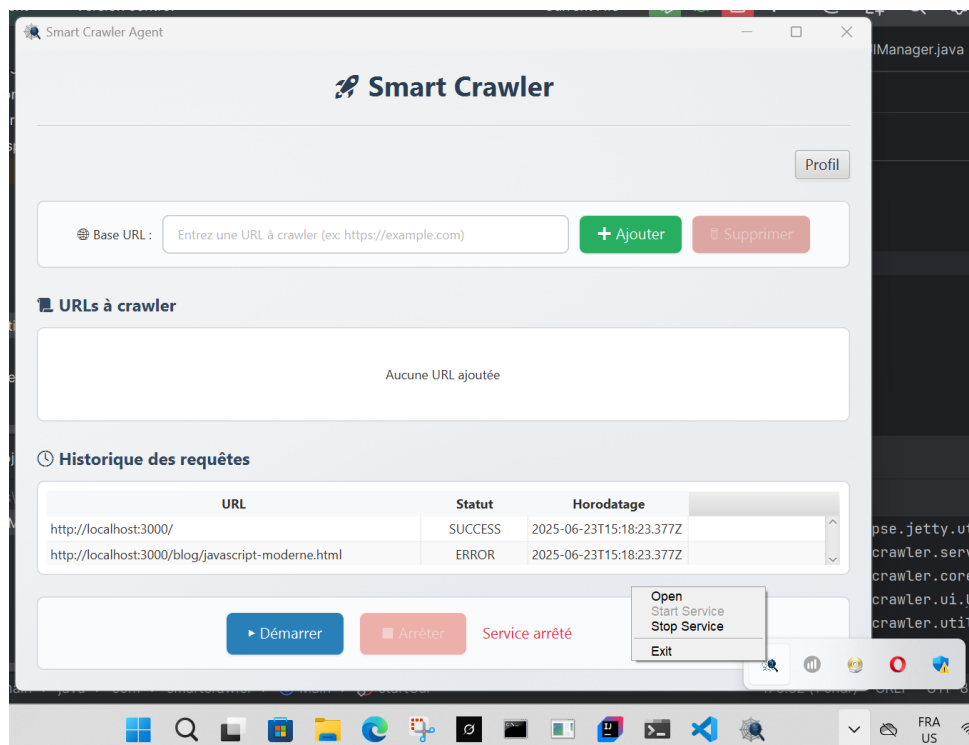


Figure 1.2: Processus complet de l'agent d'exploration

CHAPTER 2

MODELISATION MATHEMATIQUE

2.0.1 Introduction

Pour bien comprendre la suite, il faut être capable de pouvoir définir les concepts de recherche et de moteur de recherche. La recherche consiste en les actions qui ont pour but de rassembler des données suivant des critères spécifiques. En informatique et dans le monde en général, la quantité de données disponibles est en continuelle croissance. Le moteur de recherche est cet outil qui permet donc de naviguer à travers cet océan d'informations.

2.0.2 Problématique du problème de recherche

Pour se fixer les idées, imaginons que nous avons 7 tiroirs contenant des tonnes de documents. Pour toute opération de traitement, l'opération d'obtention d'un document devient un frein considérable aux opérations suivantes. De manière plus générale, avec l'avènement de la technologie, l'ensemble des données disponibles devient de plus en plus grand. Il en ressort donc un problème naturel, comment faire pour obtenir une information pertinente en un temps humainement raisonnable et pourquoi pas obtenir aussi des informations liées au centre d'intérêt ?

2.0.3 Du problème à la solution mathématique

La problématique énoncée plus haut est au cœur de la recherche. Ainsi, la recherche consiste à naviguer à travers un flux massif de données afin de sélectionner les informations qui sont sémantiquement liées à celle dont on a besoin. En regardant des informations comme des nœuds et les liens entre ceux-ci comme des arcs dirigés, le niveau de pertinence

est déterminé par le nombre de liens allant vers le nœud. Ainsi, en organisant les informations sous la forme d'un graphe dirigé partant d'un point initial, un automate pourrait déterminer comment accéder à une information et juger de son degré de pertinence. Il pourrait aussi donner d'autres informations liées à celle-ci comme suggestion.

Graphe illustrant le model mathématique

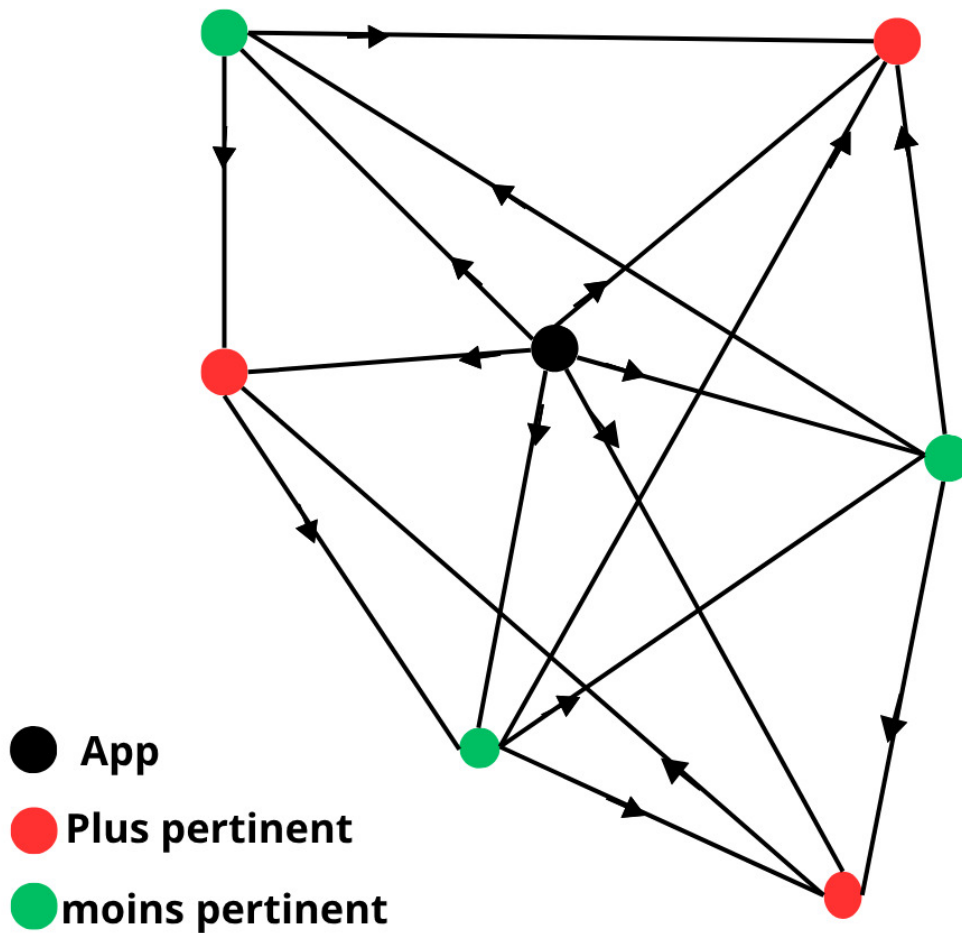


Figure 2.1: Graphe illustrant le modèle mathématique

2.0.4 De la solution mathématique au monde informatique

Dans le monde informatique, les automates parcourant les graphes dirigés sont des moteurs de recherches. Dans le cadre de la recherche et celui de notre TP particulièrement, on utilisera un graphe dirigé pondéré multi-connecté comme illustré plus haut. Celui-ci présente les avantages suivants :

- Tout sommet peut être atteint depuis tout autre sommet via des chemins orientés.
- Exprime efficacement la relation de dépendances mutuelles entre deux nœuds (information).
- Chaque nœud possède une valeur ou un poids associé permettant de jauger de la pertinence de l'information. Cela permet aux moteurs de recherche de faire un tri parmi les informations pouvant être retourné.

Quelques systèmes implémentant de la solution choisie sont : l'algorithme de recherche de Google basé sur le PageRank.

2.0.5 Conclusion

Au terme de cette modélisation mathématique, nous avons démontré que la problématique de la recherche est l'acquisition d'une donnée en temps humainement raisonnable dans un ensemble de données relativement très grand. Elle englobe aussi la pertinence des pages (information) et les relations liées à l'objet recherché (zone de l'utilisateur et zone de la recherche). Le modèle proposé (celui des graphes pondérés multi-connectés) est très utilisé dans le domaine de l'informatique par des géants tels que Google et constitue une bonne approche de par les avantages soulignés ci-dessus.

3.1 Expression des besoins fonctionnels

- Inscrire un utilisateur
- Fédérer aux autres services
- Permettre d'effectuer des recherches
- Se connecter en tant qu'utilisateur

3.2 Expression des besoins non fonctionnels

- Performance
- Scalabilité
- Sécurité
- Responsivité
- Fiabilité

3.3 Diagramme d'Analyse

3.3.1 Diagramme de contexte

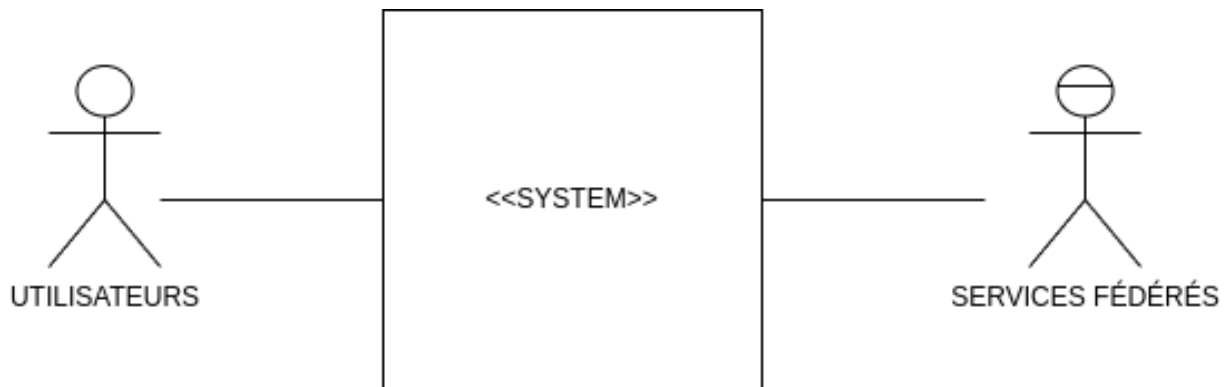


Figure 3.1: Diagramme de contexte

3.3.2 Diagramme de cas d'utilisation

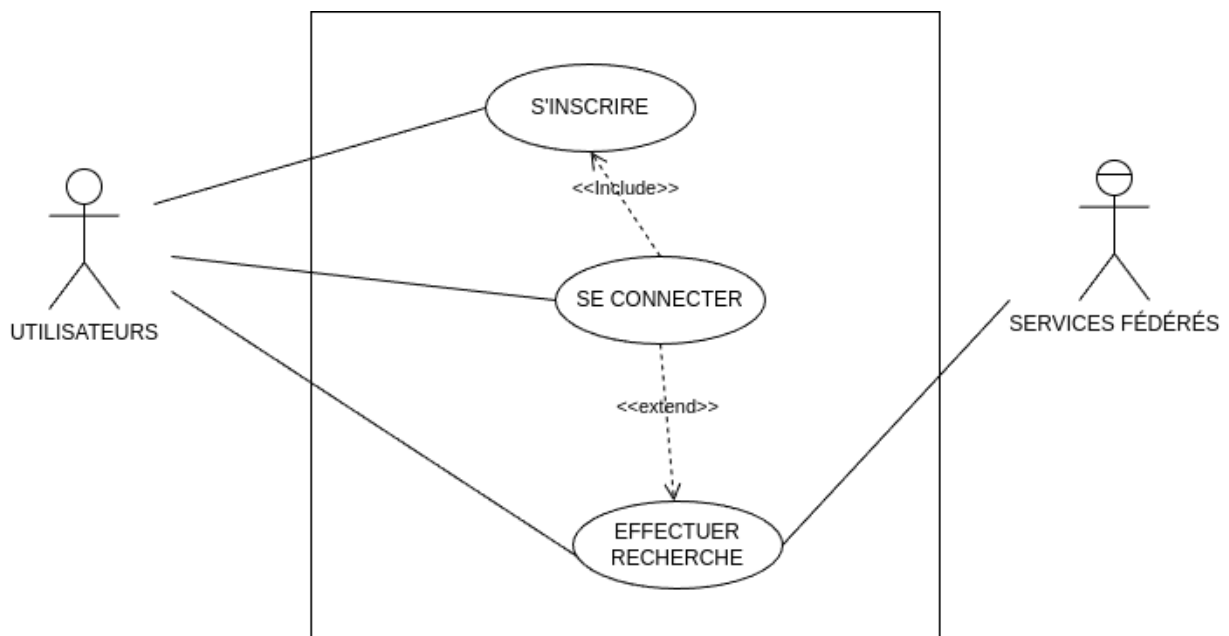


Figure 3.2: Diagramme de cas d'utilisation

3.3.3 Diagramme de séquence système des différents cas d'utilisation

S'inscrire

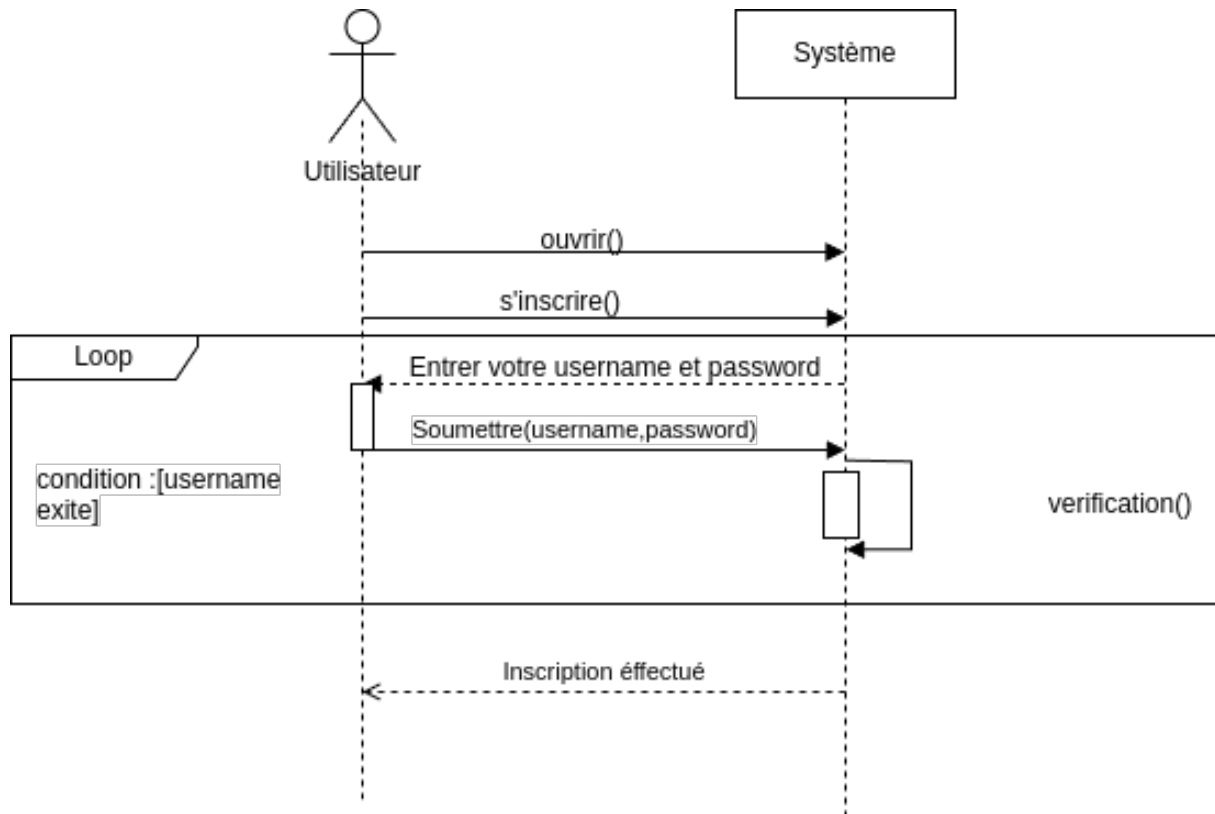


Figure 3.3: Diagramme de séquence système de s'inscrire

Se connecter

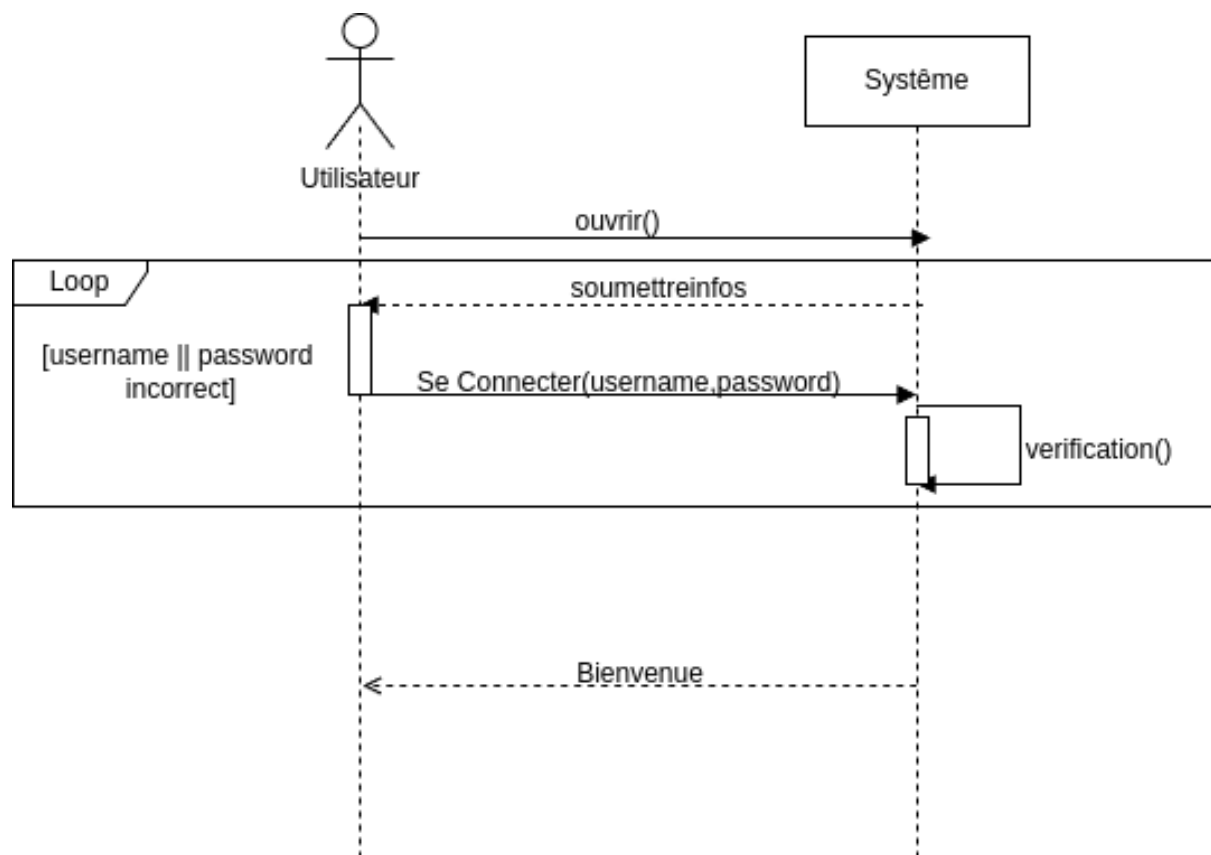


Figure 3.4: Diagramme de séquence système de se connecter

Effectuer recherche

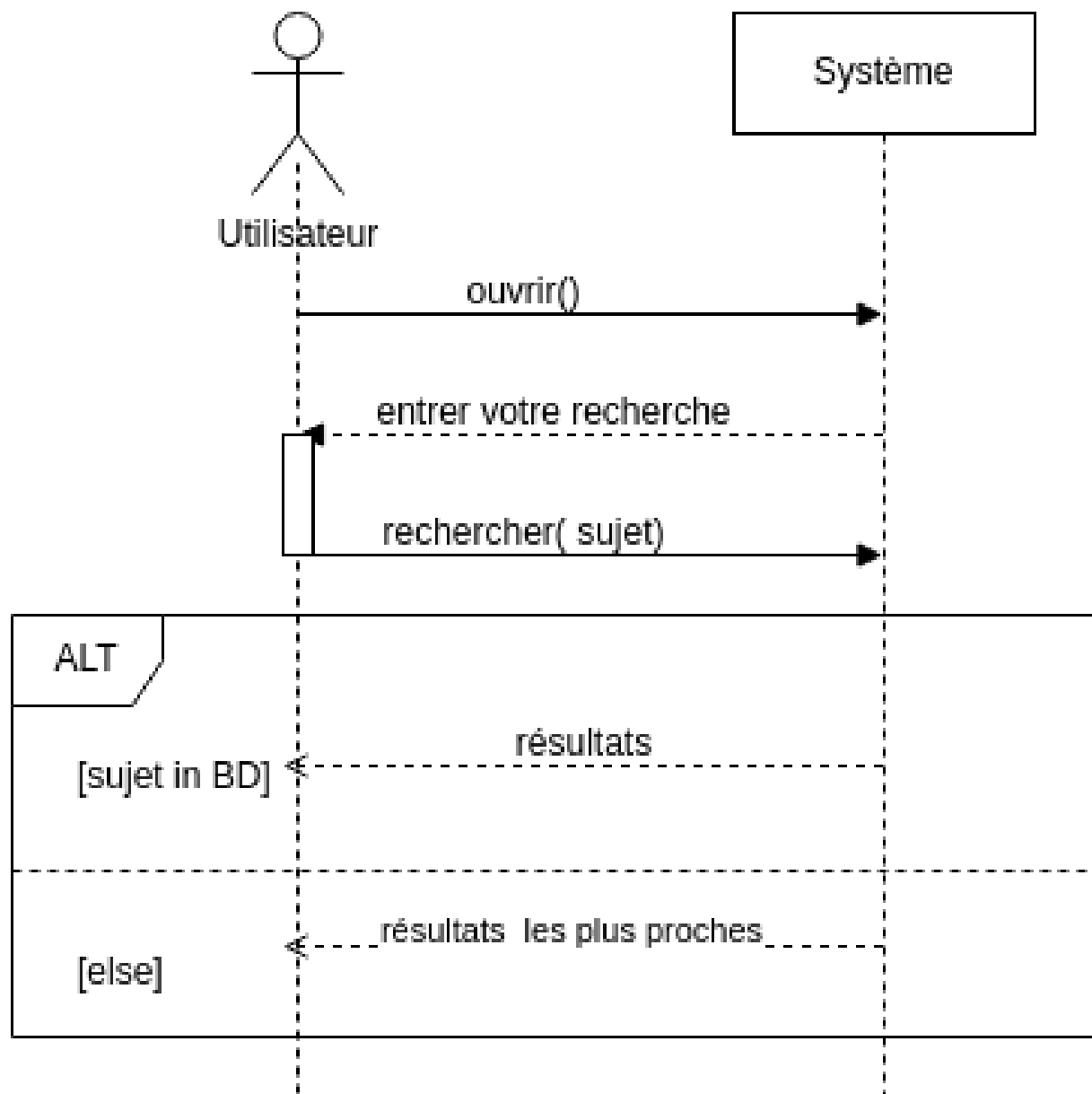


Figure 3.5: Diagramme de séquence système d'effectuer recherche

4.1 Diagrammes de conception

4.1.1 Diagramme de séquence technique

S'inscrire

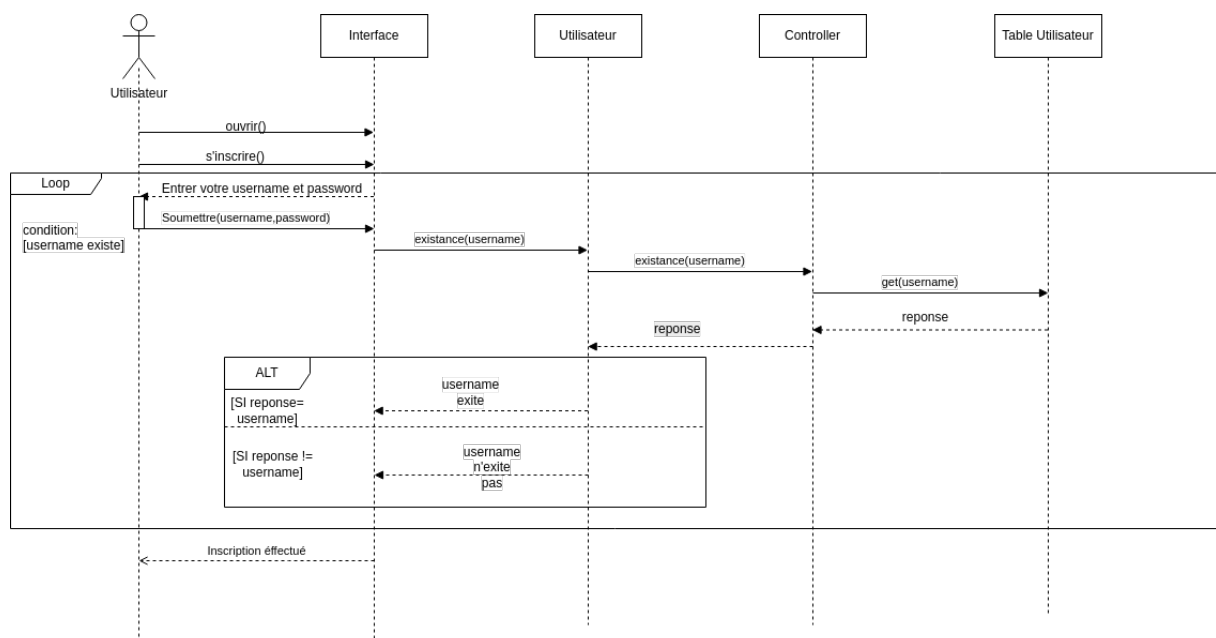


Figure 4.1: Diagramme de séquence technique de s'inscrire

Se connecter

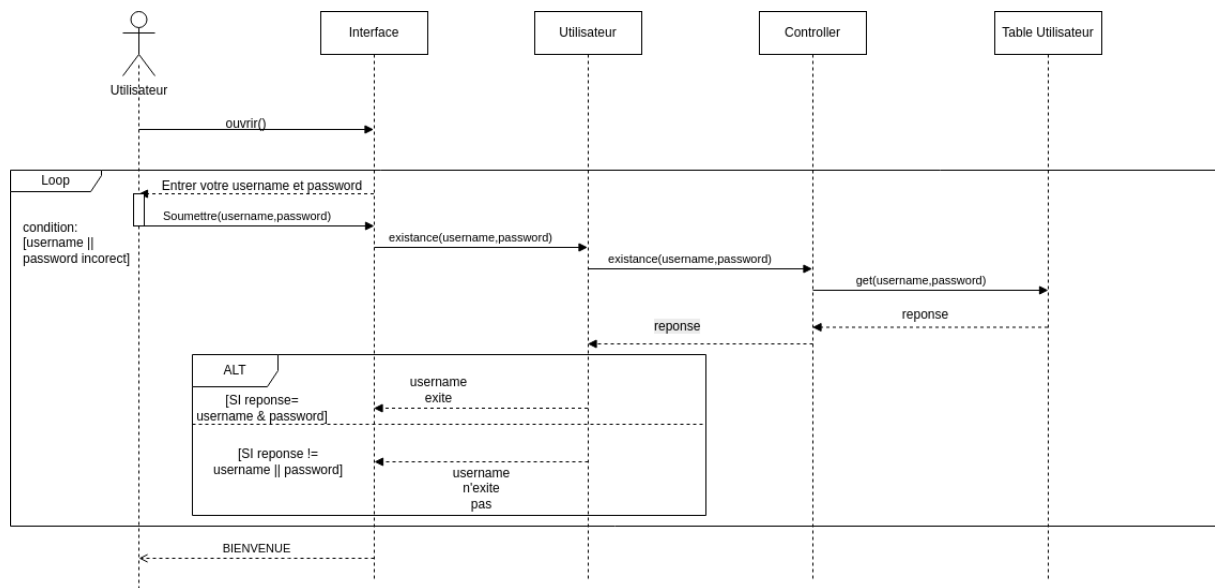


Figure 4.2: Diagramme de séquence technique de se connecter

Effectuer recherche

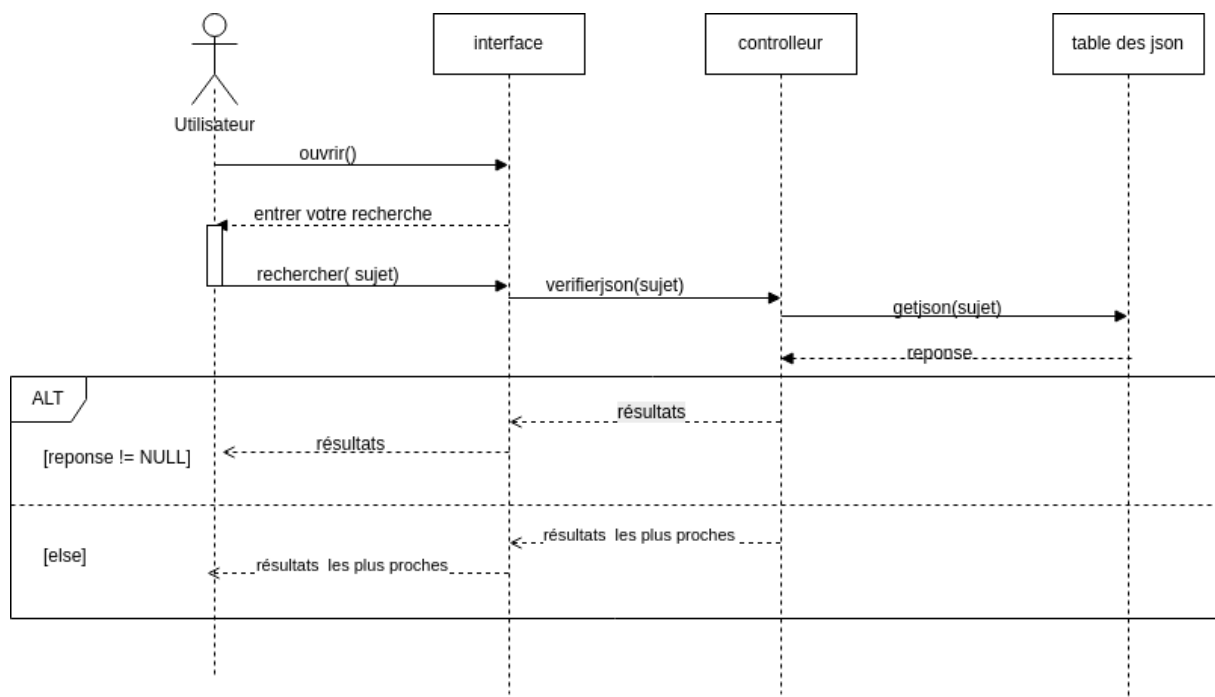


Figure 4.3: Diagramme de séquence technique d'effectuer recherche

4.2 Prise en main d'Elasticsearch

Elasticsearch est un moteur de recherche et d'analyse distribué, basé sur Apache Lucene. Il permet de stocker, rechercher et analyser de grandes quantités de données quasi en temps réel. Cette section présente les étapes pratiques pour prendre en main Elasticsearch dans le cadre de la mise en œuvre d'un moteur de recherche.

4.2.1 Installation d'Elasticsearch et de Kibana

Sur Linux (Ubuntu)

1. Mettre à jour le système :

```
sudo apt update
```

2. Installer Java (Elasticsearch nécessite Java) :

```
sudo apt install openjdk-17-jdk
```

3. Importer la clé GPG officielle :

```
wget -q0 - https://artifacts.elastic.co/GPG-KEY-elasticsearch | sudo apt-key
```

4. Ajouter le dépôt Elastic :

```
sudo apt install apt-transport-https  
echo "deb https://artifacts.elastic.co/packages/8.x/apt stable main" |  
sudo tee -a /etc/apt/sources.list.d/elastic-8.x.list
```

5. Installer Elasticsearch :

```
sudo apt update  
sudo apt install elasticsearch
```

6. Démarrer et activer Elasticsearch :

```
sudo systemctl enable elasticsearch  
sudo systemctl start elasticsearch
```

Sur Windows

1. Télécharger Elasticsearch depuis le site officiel : [magentahttps://www.elastic.co/downloads/elasticsearch](https://www.elastic.co/downloads/elasticsearch)
2. Extraire l'archive ZIP dans un dossier.
3. Exécuter Elasticsearch via PowerShell :

```
.\bin\elasticsearch.bat
```

Installation de Kibana

Kibana est l'interface graphique qu'on peut utiliser pour visualiser les données d'Elasticsearch.

1. Télécharger Kibana : [magentahttps://www.elastic.co/downloads/kibana](https://www.elastic.co/downloads/kibana)
2. Décompresser et lancer :

```
.\bin\kibana.bat (sous Windows)
sudo systemctl start kibana (sous Linux)
```

3. Accéder à Kibana : [magentahttp://localhost:5600](http://localhost:5600)

4.2.2 Création et indexation de documents JSON

Dans Elasticsearch, les données sont indexées sous forme de documents JSON.

Exemple de création d'un index avec CURL

```
PUT /bibliotheque
{
  "mappings": {
    "properties": {
      "titre": { "type": "text" },
      "auteur": { "type": "keyword" },
      "annee": { "type": "date" },
      "resume": { "type": "text" }
    }
  }
}
```

Ajout de documents

```
POST /bibliotheque/_doc/1
{
  "titre": "Le Petit Prince",
  "auteur": "Antoine de Saint-Exupéry",
  "annee": "1943-01-01",
  "resume": "Un conte poétique et philosophique."
}
```

Recherche simple

```
GET /bibliotheque/_search?q=prince
```

4.2.3 Intégration avec un projet Java Spring Boot

Pour intégrer Elasticsearch à une application Spring Boot, il est recommandé d'utiliser le client Spring Data Elasticsearch.

Dépendances Maven

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-data-elasticsearch</artifactId>
</dependency>
```

Configuration de l'URL du serveur

Dans le fichier `application.yml` :

```
spring:
  data:
    elasticsearch:
      client:
        reactive:
          endpoints: localhost:9200
```

Définition d'un modèle

```
@Document(indexName = "documents")
@Data
@JsonIgnoreProperties(ignoreUnknown = true)
public class SearchDocument {

    @Id
    private String url;
    private String title;
    private String content;
    private String description;

    @JsonDeserialize(using = DocumentTypeDeserializer.class)
    @JsonProperty("documentType")
    private DocumentType documentType;

    private List<String> links = new ArrayList<>();
    private List<String> mediaUrls = new ArrayList<>();
    private List<Map<String, String>> apiEndpoints = new ArrayList<>();

    @JsonProperty("isDynamic")
    private boolean isDynamic;

    @JsonProperty("crawlTimestamp")
    @JsonFormat(shape = JsonFormat.Shape.STRING, pattern = "yyyy-MM-dd'T'HH:mm:ss.SSS")
    private Instant crawlTimestamp;

    private int httpStatus;
    private Map<String, String> metadata = new HashMap<>();
    private String contentHash;
}
```

Création d'un repository

```
public interface LivreRepository extends ElasticsearchRepository<Livre, String> {
    List<Livre> findByTitreContaining(String motCle);
}
```

4.2.4 Indexation des données à partir d'une base existante

Il est possible d'extraire les données d'une base relationnelle, de les transformer en objets Java, puis de les indexer dans Elasticsearch :

```
@Autowired
private LivreRepository livreRepository;

public void indexerDonnees() {
    List<Livre> livres = livreDao.findAll(); // depuis une base SQL
    livreRepository.saveAll(livres);
}
```

4.2.5 Configuration des mappings

Les *mappings* définissent la structure des documents d'un index : les noms de champs, leurs types (text, keyword, date, etc.), les analyzers utilisés, et d'autres propriétés.

Exemple de mapping personnalisé

```
PUT /articles
{
  "mappings": {
    "properties": {
      "titre": {
        "type": "text",
        "analyzer": "standard"
      },
      "categorie": {
        "type": "keyword"
      },
      "contenu": {
        "type": "text",
        "analyzer": "french"
      },
      "date_publication": {
        "type": "date"
      }
    }
  }
}
```

```
}  
}
```

Les champs de type `text` sont analysés (tokenisés), tandis que ceux de type `keyword` sont indexés sans traitement, utiles pour les identifiants ou catégories.

4.2.6 Requêtes de recherche

Elasticsearch offre une syntaxe de requêtage puissante avec le DSL (Domain Specific Language) JSON.

Recherche par correspondance exacte

```
GET /articles/_search  
{  
  "query": {  
    "match": {  
      "contenu": "moteur de recherche"  
    }  
  }  
}
```

Recherche booléenne avancée

```
GET /articles/_search  
{  
  "query": {  
    "bool": {  
      "must": [  
        { "match": { "contenu": "elasticsearch" } },  
        { "term": { "categorie": "informatique" } }  
      ],  
      "filter": [  
        { "range": { "date_publication": { "gte": "2024-01-01" } } }  
      ]  
    }  
  }  
}
```

Recherche avec score de pertinence

```
GET /articles/_search
{
  "query": {
    "multi_match": {
      "query": "analyse des données",
      "fields": ["titre^2", "contenu"]
    }
  }
}
```

Le champ `titre` est ici pondéré avec un facteur de score supérieur (grâce au `^2`).

4.2.7 Requêtes avancées et score de pertinence

Elasticsearch permet d'effectuer des recherches booléennes, avec pondération :

```
POST /bibliotheque/_search
{
  "query": {
    "bool": {
      "must": [
        { "match": { "resume": "philosophique" } },
        { "match": { "titre": "prince" } }
      ]
    }
  }
}
```

4.2.8 Utilisation de Kibana pour visualiser les données

- Dans Kibana, on peut visualiser les index, les documents, et créer des visualisations (graphiques, tableaux).
- Dev Tools permet d'exécuter des requêtes de test.
- Lens permet de créer des dashboards dynamiques.

4.2.9 Sécurité et authentification (optionnel)

Elastic Stack propose des fonctions de sécurité comme :

- Authentification par utilisateur.
- Attribution de rôles (lecture, écriture).
- Chiffrement TLS des communications.

4.3 Services fédérés

Les API ou services externes qui seront intégrés sont :

1. **Projet 2** : Portail Web Responsive pour APIs
2. **Projet 3** : Bug Tracker
3. **Projet 4** : PoI (Point d'Intérêt)
4. **Projet 5** : Business Book
5. **Projet 6** : Pricing API
6. **Projet 7** : Ride and Go
7. **Projets 8-9** : Fare Calculator (Web & Mobile)
8. **Projet 10** : Organisation API Optimisée
9. **Projets 11-15** : Optimisation Système
10. **Projet 16** : Agence de Location de Véhicules
11. **Projet 18** : Agence de Voyage - Bus Station
12. **Projet 21** : API de Géofence
13. **Projet 22** : Service d'Éducation
14. **Projet 23** : Notification API
15. **Projet 24** : Feedback API
16. **Projet 25** : Service de Communication
17. **Projet 26** : Gestion des Syndicats

18. **Projet 27** : Gestion Driving School
19. **Projet 28** : Flashop App
20. **Projet 29** : Plateforme Chauffeurs Freelance
21. **Projet 30** : Optimisation Gestion de Flotte
22. **Projet 31** : Service de Gestion de Médias
23. **Projet 32** : Optimisation des Performances Réseau
24. **Projet 33** : Classification Automatique du Trafic Réseau
25. **Projet 34** : Répartition du Trafic WAN

CHAPTER 5

DÉVELOPPEMENT ET IMPLEMENTATION

5.1 Environnement de développement

Dans le cadre de ce projet, nous avons utilisé comme environnement de développement :

- **Les IDE** : IntelliJ, Visual Studio
- **Le serveur** : Elasticsearch présent sur notre machine
- **Les frameworks** : Spring Boot, React, Tailwind

5.2 Développement du frontend responsive

Pour développer un frontend responsive, nous avons codé en JavaScript (avec la bibliothèque React) et en CSS (avec Tailwind).



Figure 5.1: Interface utilisateur développée avec React

5.3 Développement du backend

Pour développer le backend de notre application, nous avons codé en Java (avec la bibliothèque Spring Boot).



Figure 5.2: Architecture backend avec Spring Boot

5.4 Intégration de l'inscription utilisateur

Afin de sécuriser l'accès à notre moteur de recherche, une couche d'authentification a été intégrée à l'architecture de l'application. Celle-ci permet de contrôler l'accès aux différentes fonctionnalités du système et de garantir que seuls les utilisateurs autorisés peuvent interagir avec la plateforme.

Objectifs de l'authentification

- Restreindre l'accès au tableau de bord de gestion.
- Protéger les endpoints de l'API exposée par le backend.
- Permettre une gestion différenciée des rôles utilisateurs.

Technologies utilisées

- **Spring Security** : pour la configuration de l'authentification côté backend.
- **JWT (JSON Web Token)** : pour la gestion des sessions et la sécurisation des échanges.
- **bcrypt** : pour le hachage des mots de passe.

Fonctionnement

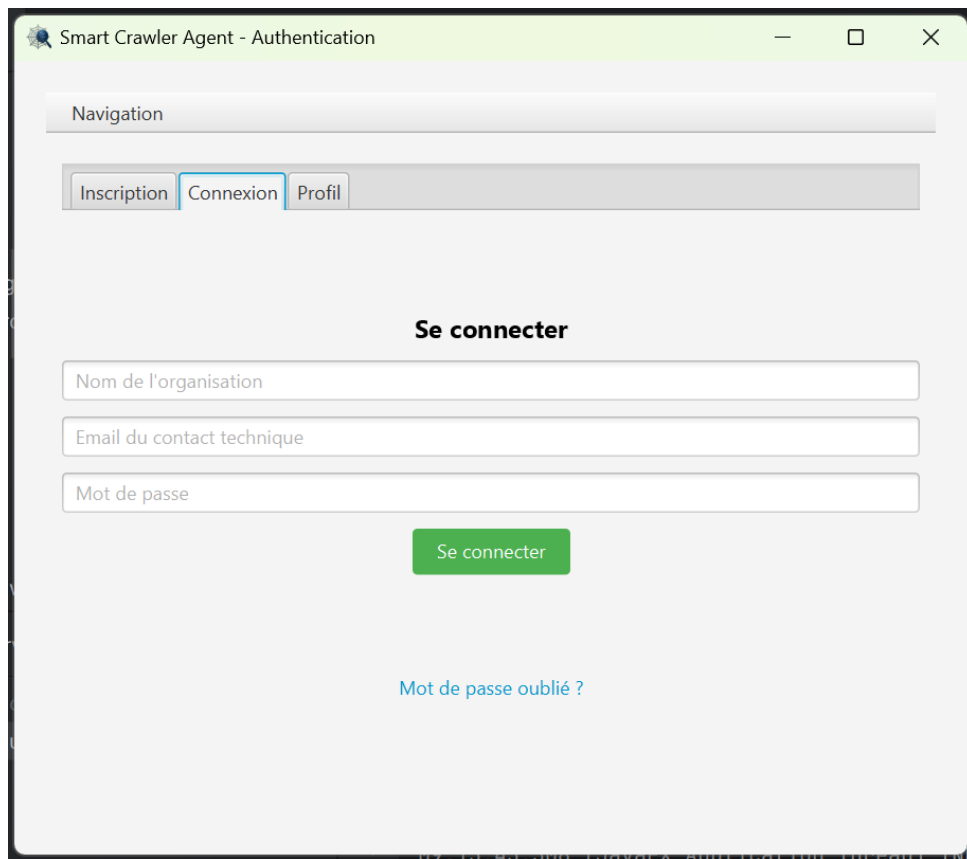
Lorsqu'un utilisateur tente de se connecter :

1. Le frontend React envoie les identifiants (email, mot de passe) via une requête POST.
2. Le backend Spring Boot vérifie les identifiants avec la base de données.
3. En cas de succès, un token JWT est généré et renvoyé au client.
4. Le client stocke ce token (dans le *localStorage* ou les cookies) et l'utilise pour toutes les requêtes suivantes.
5. Le backend vérifie le token à chaque requête entrante avant de traiter la demande.

6.1 Scénarios de test

6.1.1 Interface d'inscription

L'interface d'inscription permet aux utilisateurs de créer un compte en fournissant leurs informations personnelles.

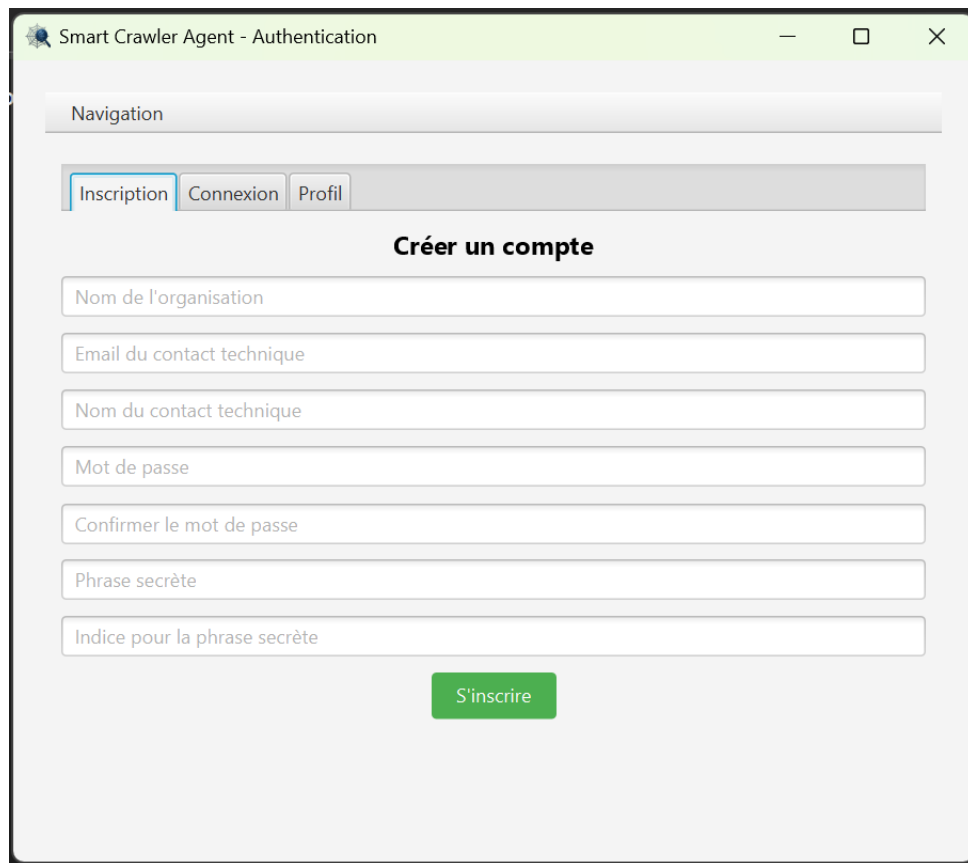


The screenshot displays a web browser window titled "Smart Crawler Agent - Authentication". The interface features a navigation bar with three tabs: "Inscription", "Connexion", and "Profil". The "Connexion" tab is currently selected. Below the navigation bar, the heading "Se connecter" is centered. Underneath, there are three input fields: "Nom de l'organisation", "Email du contact technique", and "Mot de passe". A green button labeled "Se connecter" is positioned below the input fields. At the bottom of the form, there is a blue link that reads "Mot de passe oublié ?".

Figure 6.1: Interface d'inscription utilisateur

6.1.2 Interface de connexion

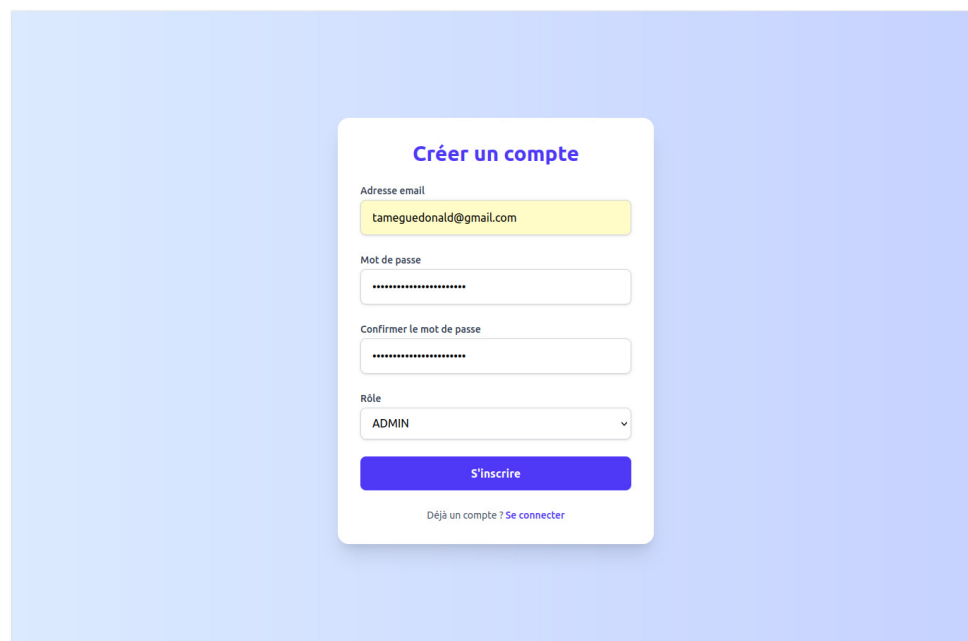
L'interface de connexion permet aux utilisateurs enregistrés d'accéder à la plateforme avec leurs identifiants.



The image shows a web browser window titled "Smart Crawler Agent - Authentication". The interface has a navigation bar with three tabs: "Inscription" (selected), "Connexion", and "Profil". Below the navigation bar, the heading "Créer un compte" is centered. The form contains the following fields: "Nom de l'organisation", "Email du contact technique", "Nom du contact technique", "Mot de passe", "Confirmer le mot de passe", "Phrase secrète", and "Indice pour la phrase secrète". A green "S'inscrire" button is positioned at the bottom of the form.

Figure 6.2: Interface de connexion

6.1.3 Interface d'authentification pour utilisateur



The image shows a web browser window titled "Smart Crawler Agent - Authentication". The interface has a navigation bar with three tabs: "Inscription" (selected), "Connexion", and "Profil". Below the navigation bar, the heading "Créer un compte" is centered. The form contains the following fields: "Adresse email" (pre-filled with "tameguedonald@gmail.com"), "Mot de passe" (masked with dots), "Confirmer le mot de passe" (masked with dots), "Rôle" (dropdown menu with "ADMIN" selected), and a blue "S'inscrire" button. At the bottom of the form, there is a link: "Déjà un compte ? Se connecter".

Figure 6.3: inscription

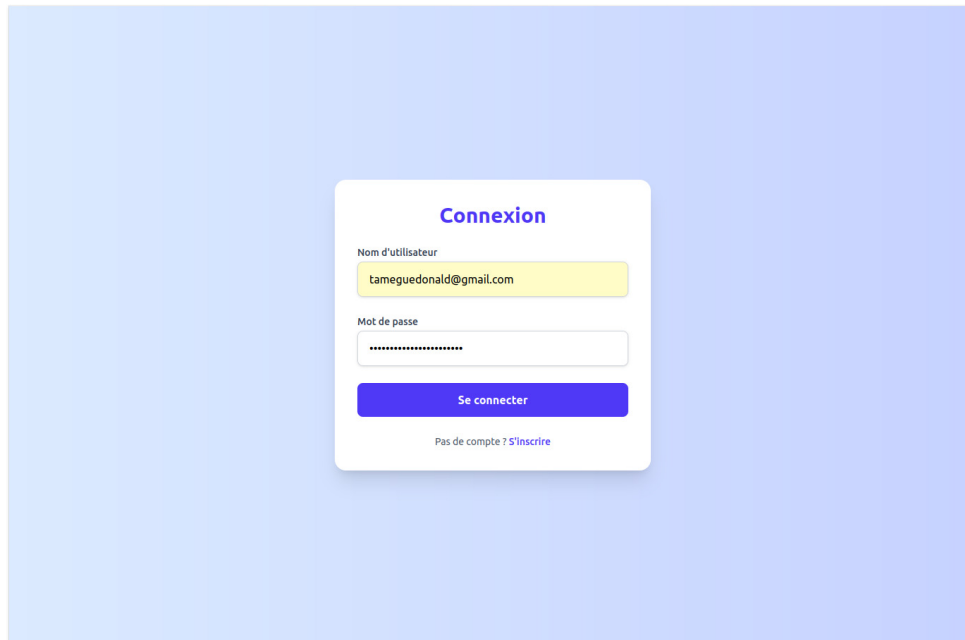


Figure 6.4: connexion

6.1.4 Interface de recherche

L'interface de recherche offre une barre de recherche intuitive pour saisir les requêtes et afficher les résultats.

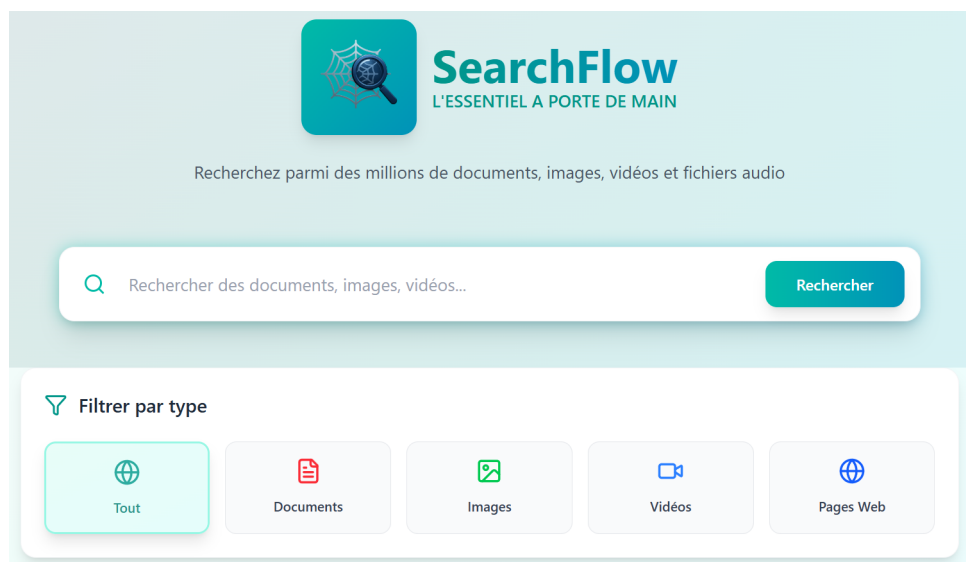


Figure 6.5: Interface de la page de recherche

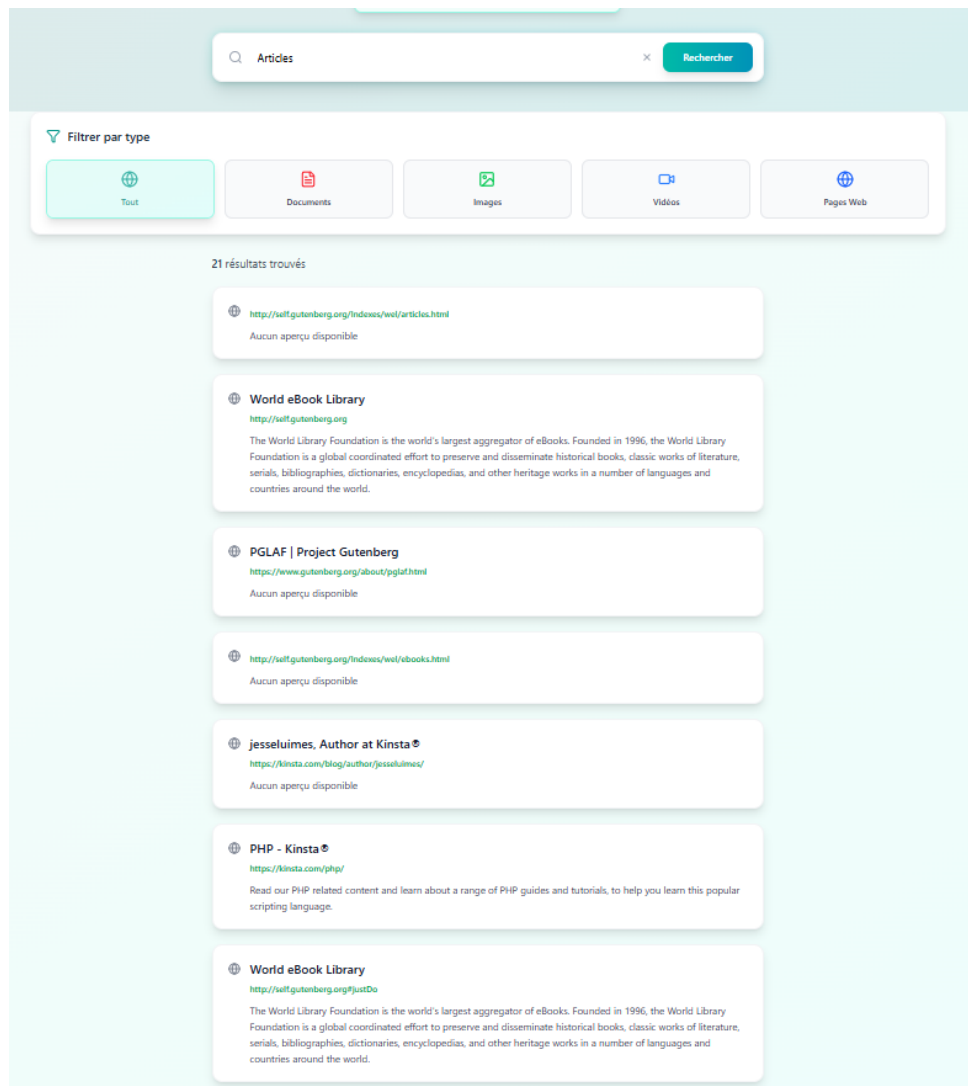


Figure 6.6: Affichage des résultats de recherche

6.1.5 Acceptation des termes

Avant l'inscription, les utilisateurs doivent accepter les termes et conditions d'utilisation.

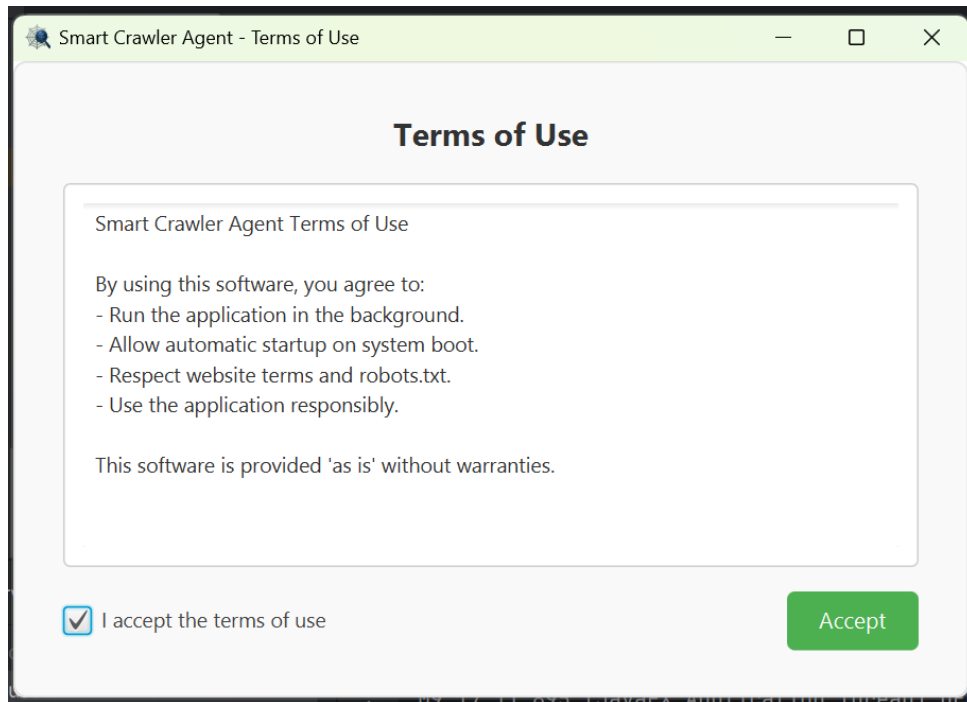


Figure 6.7: Interface d'acceptation des termes

6.1.6 Fonctionnalité d'autocomplétion

Une fonctionnalité d'autocomplétion a été intégrée pour suggérer des termes de recherche pertinents lors de la saisie.

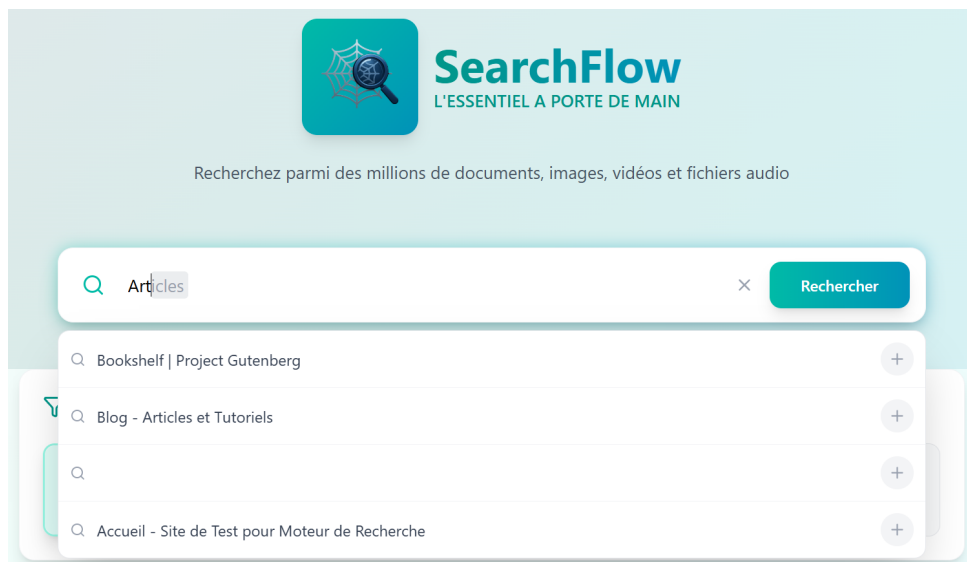


Figure 6.8: Fonctionnalité d'autocomplétion dans la barre de recherche

6.2 Pertinence de la recherche

La pertinence des résultats de recherche repose sur un calcul de scores combinant plusieurs facteurs. Trois composants principaux ont été implémentés :

- **Fraîcheur (FreshnessScorer)** : Évalue la récence des documents à l'aide d'une fonction de décroissance exponentielle. Les documents récents (moins de 24 heures) bénéficient d'un boost, tandis que ceux datant de plus de 30 jours voient leur score diminuer progressivement.
- **Popularité (PopularityScorer)** : Calcule un score basé sur des métriques comme le nombre de liens entrants, les partages, les commentaires et l'autorité du domaine. Une fonction sigmoïde normalise le score pour éviter les valeurs extrêmes.
- **Localisation (LocationScorer)** : Ajuste le score en fonction de la distance géographique entre l'utilisateur et la localisation du document, avec une portée maximale de 1000 km.

Ces scores sont agrégés avec le score natif d'Elasticsearch (basé sur la correspondance textuelle) pour produire un score final, garantissant des résultats à la fois pertinents et contextuels.

CONCLUSION GÉNÉRALE

À l'ère du numérique, l'accès rapide, pertinent et centralisé à l'information constitue un enjeu fondamental pour les utilisateurs. Ce projet a permis de concevoir et de mettre en œuvre un moteur de recherche responsive intégré à une application fédérant plusieurs services, tout en assurant une authentification sécurisée des utilisateurs.

Grâce à l'utilisation de technologies modernes telles qu'Elasticsearch, Spring Boot et ReactJS, nous avons pu répondre efficacement à la problématique initiale : simplifier l'accès à des ressources multiples, tout en garantissant la pertinence des résultats, la sécurité des données, et une expérience utilisateur fluide. L'architecture mise en place assure également une bonne évolutivité, rendant possible l'intégration de nouveaux services à l'avenir.

Ce projet nous a permis non seulement d'appliquer nos connaissances en conception d'applications distribuées, en développement web et en gestion de données, mais aussi de nous confronter aux enjeux concrets de performance, d'ergonomie et de sécurité dans un environnement fédéré. Il ouvre la voie à des améliorations futures, telles que l'implémentation de filtres de recherche avancés, de recommandations personnalisées ou encore d'un tableau de bord d'analyse des requêtes.

Difficultés rencontrées

Comme difficultés rencontrées, nous avons :

- La difficulté principale rencontrée était la mise en place d'Elasticsearch puisque celui-ci nécessitait l'utilisation d'une grande partie de la RAM.
- L'indexation des services n'a pas été de tout repos.

Perspectives

- Inscription avec OTP pour utilisateur.
- Gérer les filtres de manière optimale.
- Améliorer l'interaction des services avec le moteur de recherche.
- Activer les filtres après consolidation de l'application.

RÉFÉRENCES BIBLIOGRAPHIQUES

- Elasticsearch : `magentahttps://www.elastic.co/guide/en/elasticsearch/reference/current/index.html`
- Spring Boot : `magentahttps://docs.spring.io/spring-boot/docs/current/reference/html/`
- Spring Security : `magentahttps://docs.spring.io/spring-security/reference/`
- React : `magentahttps://react.dev/`
- JavaScript : `magentahttps://developer.mozilla.org/en-US/docs/Web/JavaScript`
- Tailwind CSS : `magentahttps://tailwindcss.com/docs`